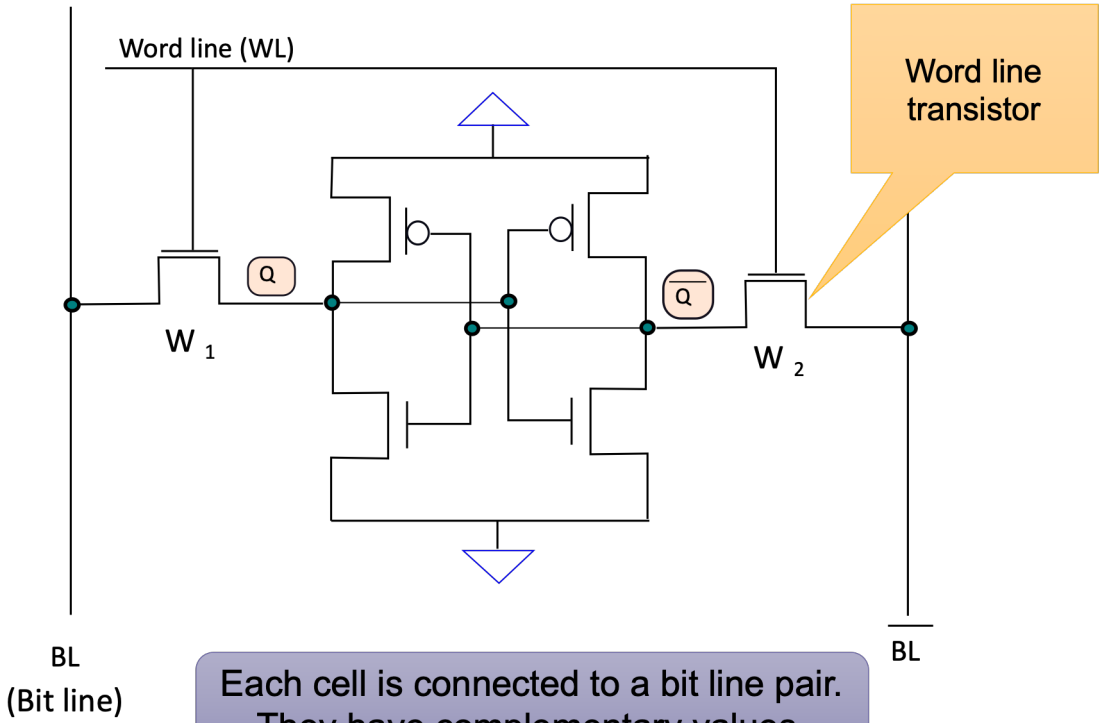
6. The Data Cache

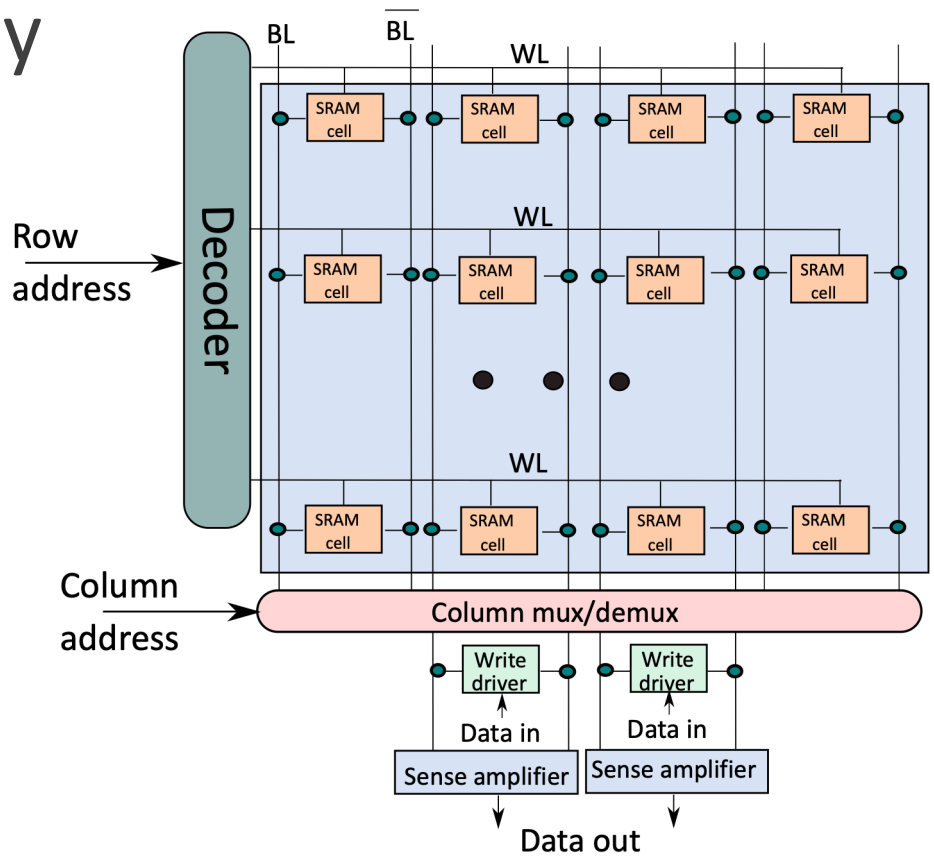
CACHE ORGANIZATION

Basic SRAM Cell in CMOS

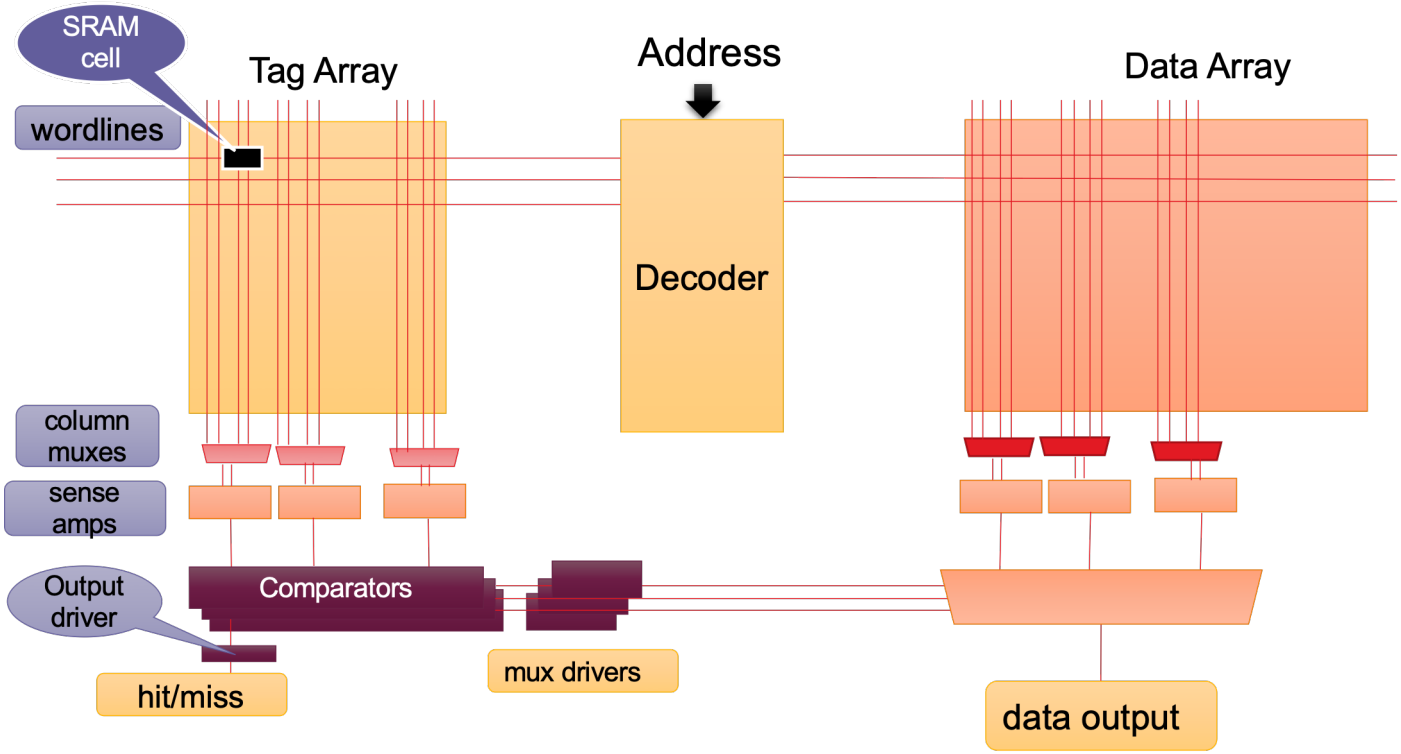


Each cell is connected to a bit line pair. They have complementary values.

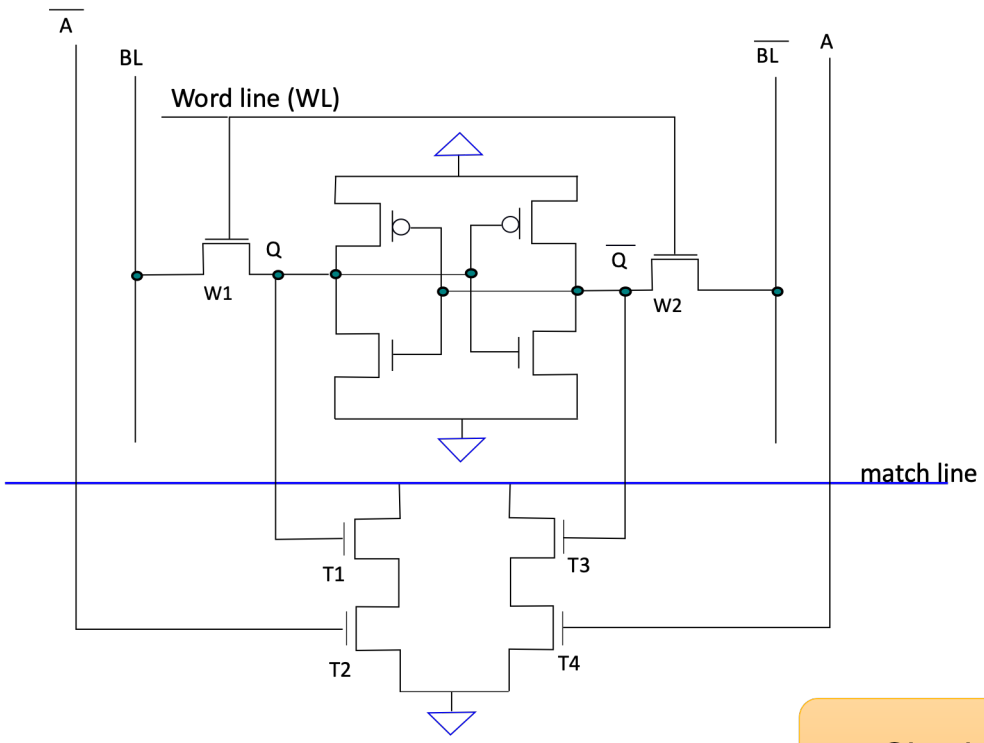
SRAM Array


BL
$\overline{BL}$
WL
WL
WL
SRAM cell
Row address
Decoder
Column address
Column mux/demux
Write driver
Data in
Sense amplifier
Data out

Structure of A Cache



A CAM Cell

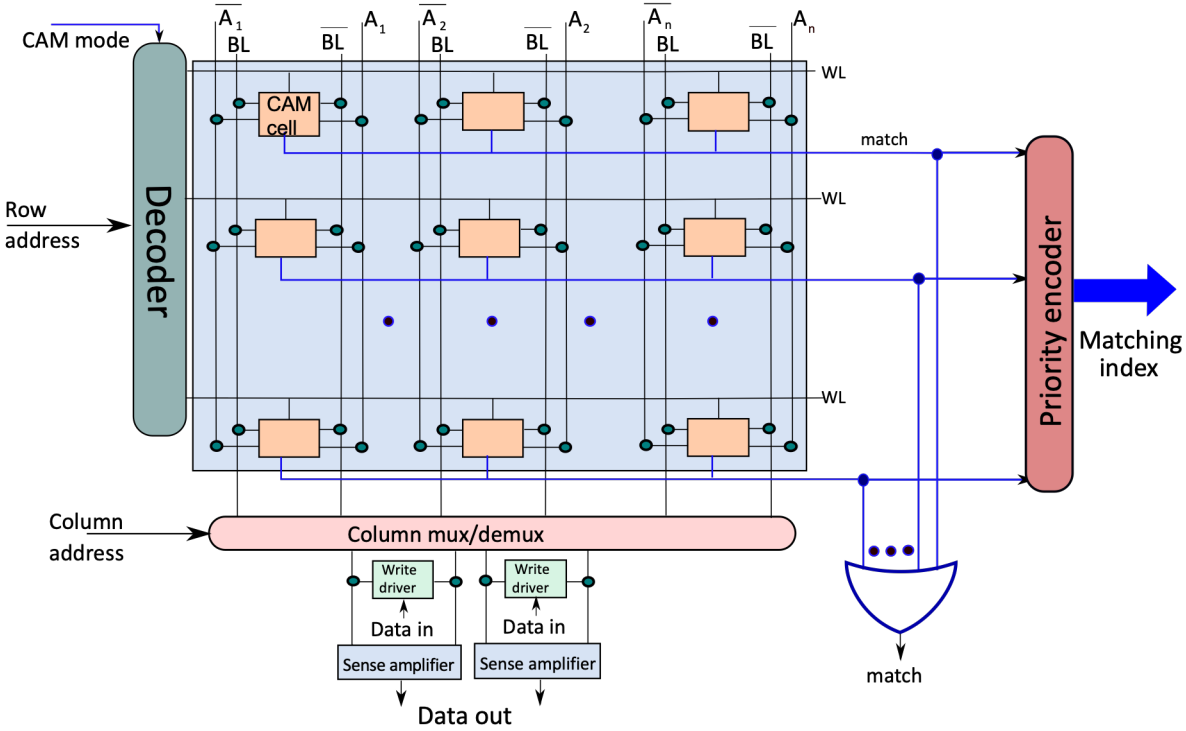


Q	$\overline{A}$	T_1	T_2
0	0	off	off
0	1	off	on
1	0	on	off
1	1	on	on

$\overline{Q}$	A	T_3	T_4
0	0	off	off
0	1	off	on
1	0	on	off
1	1	on	on

Checks whether Q = A

CAM Array


CAM mode
Row address
Decoder
CAM cell
WL
match
Priority encoder
Matching index
Column address
Column mux/demux
Write driver
Data in
Sense amplifier
Data out
match

Cache Line Replacement Policy

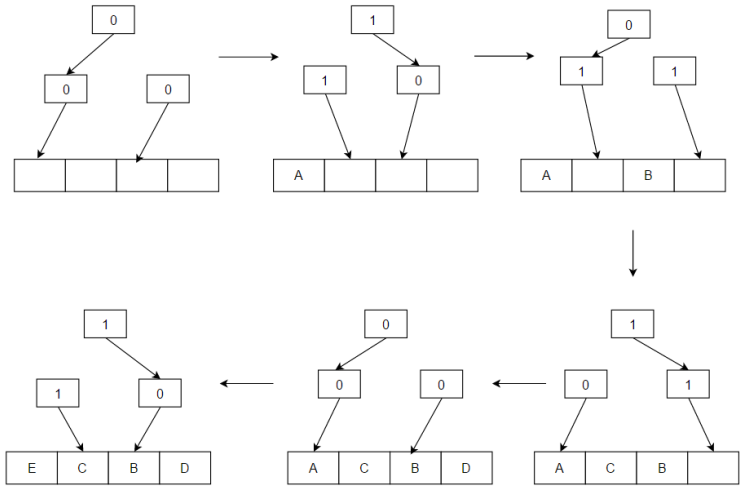
- Implementing replacement policies is difficult, even for moderate cache associativities.
  - Obviously, no problem for $A = 1$.
  - Taking true LRU as an approximation for Bélády ("provably optimal but unimplementable"), can we implement it for $A = 2$?
    - Easy enough: just keep one bit per set to track MRU/LRU.
  - How about $A = 3$? (Three bits suffice.)
  - How about $A = 4$?
  - Does this idea scale?
  - Is it even worth trying to implement true LRU, given that it is an approximation anyway?
- Here is a 6-bit encoding of all possible $4!$ possibilities ("states").
  - Bit 0: Way 1 MRU than Way 0.
  - Bit 1: Way 2 MRU than Way 0.
  - Bit 2: Way 3 MRU than Way 0.
  - Bit 3: Way 2 MRU than Way 1.
  - Bit 4: Way 3 MRU than Way 1.
  - Bit 5: Way 3 MRU than Way 2.
- Can build an FSM using this state encoding and a two-bit encoding for the current access to determine next state and victim way.
- Theoretical minimum encoding of states requires $\lceil \lg 4! \rceil = 5$ bits.

Source: https://people.computing.clemson.edu/~mark/464/p_lru.txt

Pseudo-LRU (Tree-PLRU)

By Advaitjavadekar - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=134704066

- An efficient algorithm to select an item that most likely has not been accessed very recently, given a set of items and a sequence of access events to the items.
  - Needs $A - 1$ bits per set to represent the branch points of a binary decision tree ("states").
  - Results in a simple FSM for picking the victim way on a miss (obviously, only for the case where all four ways are occupied) or updating the state on a hit to a given way.



State	Victim Way
00x	0
01x	1
1x0	2
1x1	3

Ref to way #	Next State
0	11_
1	10_
2	0_1
3	0_0

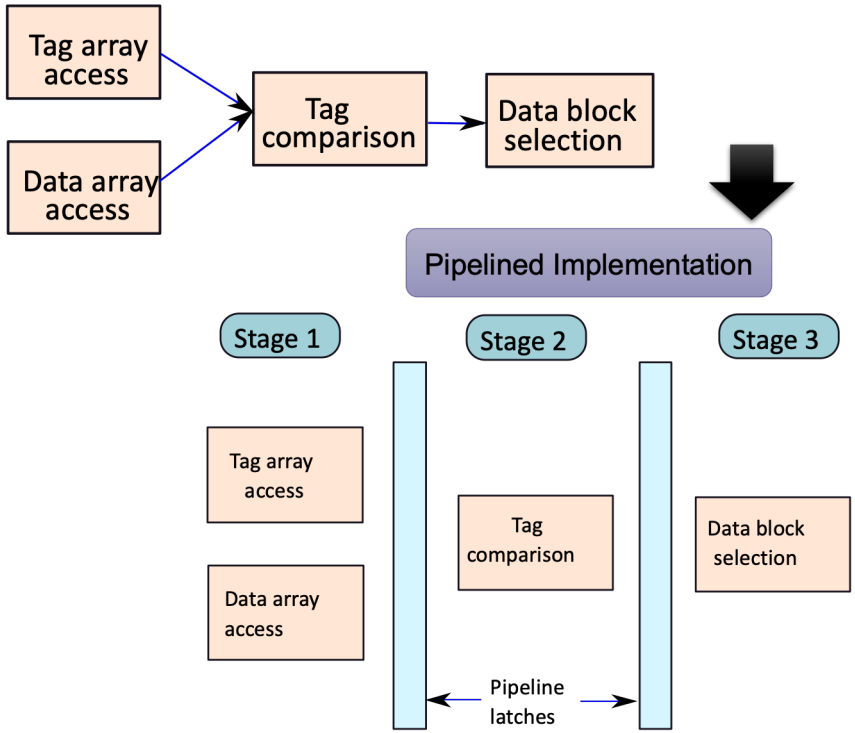
Source: https://people.computing.clemson.edu/~mark/464/p_lru.txt

Advanced Cache Design Choices

- If best-case cache access latency is still multiple processor cycles, then we can improve throughput by designing a *pipelined cache*.
- If we have a cache miss, we can avoid stalling later memory operations by designing a *non-blocking cache*.
- Add a *victim cache* to produce the illusion of greater associativity or larger capacity.
- Reduce conflict misses using a *skewed-associative cache* for caches that are on critical paths and therefore are particularly sensitive to misses.
- We can reduce tag storage without going to a longer cache line size by using a *sectored cache*.

Pipelined Cache

- Mechanism
  - Add pipeline registers.
- Read
  - Three-stage pipeline: access tag and data arrays, compare tags, select data block.
- Write
  - Three-stage pipeline: access tag array, compare tags, write in data array.



Non-Blocking Cache: Basics

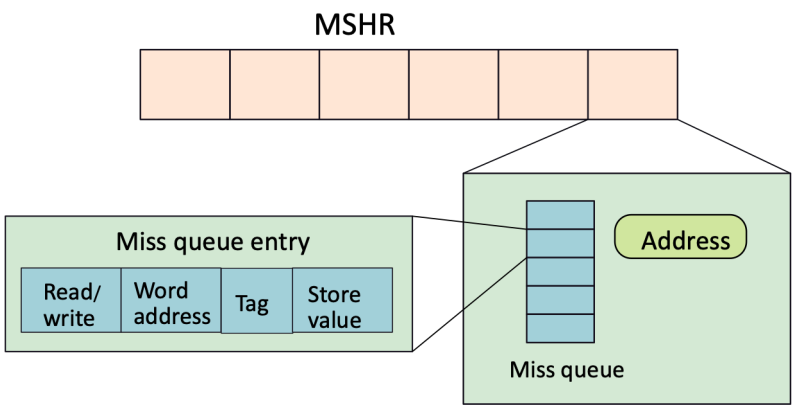
- Memory access A_1 to address $m_1 = \langle t_i, i_1, f_1 \rangle$ results in a D$ miss. Let $b_1 = \langle t_1, i_1 \rangle$ be its memory block number.
  - While this miss is being handled, a second memory access A_2 to address $m_2 = \langle t_2, i_2, f_2 \rangle$ arrives. Let $b_2 = \langle t_2, i_2 \rangle$ be its memory block number.
  - Several cases arise, depending on whether m_1 and m_2 reference the same memory block, whether access A_2 is a read or a write, and whether access A_2 is a hit or a miss.
- Implication: Memory operations may complete out of order.
- Implementing this policy requires a mechanism for recording outstanding cache misses.

R/W	$b_1 = b_2$?	H/M	Comment
R	Y	H	Cannot happen
R	Y	M	Secondary miss to b_1; cases 2a and 2b
R	N	H	Handle hit right away
R	N	M	Primary miss to b_2
W	Y	H	Cannot happen
W	Y	M	Secondary miss to b_1; case 1
W	N	H	Handle hit right away
W	N	M	Primary miss to b_2

Mechanism: Miss Status Holding Register (MSHR)

Design

- For each missed memory block, we have an associated miss queue.



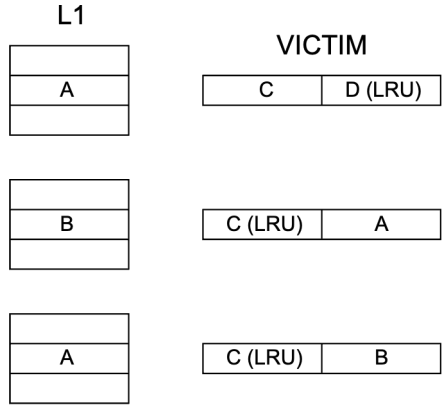
Operation

- On a *primary miss* for a memory block, an entry is created for the block in its MSHR. If there are no free entries, the request blocks.
  - We also create an entry in the miss queue of this MSHR entry.
- Another miss to the same memory block that arrives while the primary miss is being handled is called a *secondary miss*.
  1. If it is a write, we just append it to the tail of the miss queue.
  2. If it is a read, we search for earlier writes in the miss queue to the same set of bytes.
     a. If we find an entry, the value is forwarded.
     b. Else, we enqueue the entry.
- When the primary miss is handled, its miss queue is drained.

N. P. Jouppi, "Improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers," Proceedings. The 17th Annual International Symposium on Computer Architecture, Seattle, WA, USA, 1990, pp. 364-373, doi: 10.1109/ISCA.1990.134547.

Victim Cache

- A small, typically fully-associative cache placed in the refill path of a CPU cache.
  - Misses in the cache that hit in the victim cache have a one cycle penalty.
  - Any cache line evicted from the data cache is cached in the victim cache.
- Gives the illusion of greater associativity or greater capacity.
- In modern processors, often used at L3/L4.
  - Intel's Crystal Well uarch has on-package L4 D$ that is a victim cache of L3 D$.
  - AMD's Zen uarch has the L3 D$ as a victim cache, with shadow tags for snooping.
    - 3D V-Cache technology: Stacking additional, large L3 D$ directly on the CPU chip, creating massive victim caches that significantly boost performance in applications like gaming.
  - IBM POWER5 has a 4-12 MiB L3 D$ is a victim cache.



By Anishaladsaria - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=53225638

- Operation
  - D$ hit: No action.
  - D$ miss, V$ hit: Block in V$ exchanged with block evicted from D$. This V$ entry is now MRU.
  - D$ miss, V$ miss: Block is brought to D$ from next level. Block evicted from D$ is stored in V$.
- Example
  - Trace is A, B, A, B, …

Skewed-Associative Cache

- A specialized multi-way cache design that improves upon set-associative structures by reducing conflict misses without increasing hardware complexity.
  - Uses distinct indexing functions for each way, thus reducing the probability of two memory blocks conflicting in multiple ways.
  - 👍 A 2-way skewed-assoc. cache provides performance closer to a 4-way set-associative cache but has the same number of tag comparators as a 2-way SA cache.
  - 👎 Needs logic to generate different hashes for different ways.
  - 👎 Mapping of a memory block is not longer simple.
  - 👎 Replacement policy is complicated.
- Particularly effective for TLBs, BTBs, and I$, where the effect of misses is exacerbated.

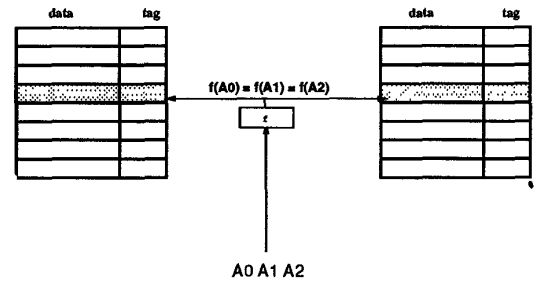


Figure 3: 3 data conflicting for a single set on a two-way set-associative cache

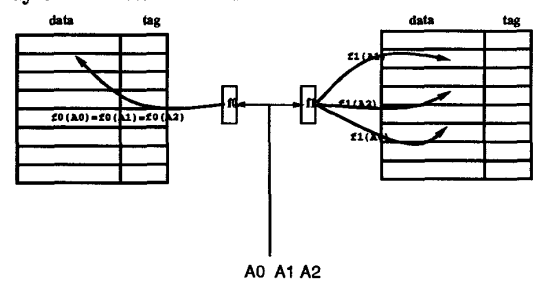


Figure 4: Data conflicting for a cache line on bank 0, but not on bank 1 on a skewed-associative cache

A. Seznec, “A case for two-way skewed-associative caches”, ISCA '93: Proceedings of the 20th annual international symposium on computer architecture (June 1993). https://doi.org/10.1145/165123.165152

Skewed-Associative Cache

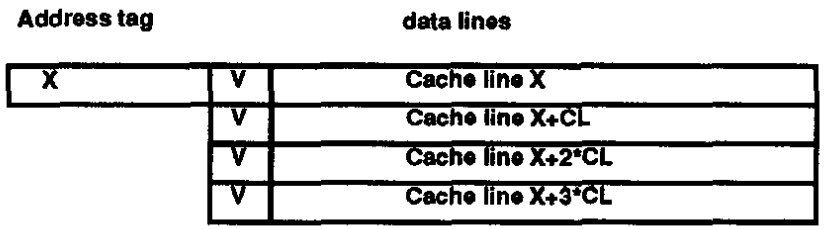
The Hash Functions

- Effectiveness depends on the hash functions.
- Desired properties
  - Equitability: The number of memory blocks mapped to each cache line must be the same.
  - Inter-way dispersion: Whenever two lines of data conflict for a single location in cache way i, they have very low probability to conflict for a location in cache way j.
  - Local dispersion in a single way: The hash functions must limit the number of conflicts when mapping any region of consecutive memory blocks in a single cache way (to respect spatial locality at a granularity larger than a cache line).

- Given:
  - $\$\left(2, B = 2^b, C = 2^c; S = 2^s\right)$
  - `A[W-1:0] = {A3[W-(2*s+b)-1:0], A2[s-1:0], A1[s-1:0], A0[b-1:0]}`
  - Integer $0 \leq T < 2^s$,
  - Integer $\overline{T} = 2^s - 1 - T$,
  - A bit permutation ϕ on s-vectors, e.g., identity, perfect shuffle, bit reversal.
- Map `A` to:
  - Set # $A1 \oplus (\phi(A2) \cdot T)$ in way 0.
  - Set # $A1 \oplus \left(\phi(A2) \cdot \overline{T}\right)$ in way 1.
- This can be done with just an extra layer of XOR gates.

Sectored Cache

- A sectored cache organizes multiple cache lines into “sectors” that are managed by a single address tag but can be validated independently.
  - Reduces tag storage overhead while allowing cache line transfers to reduce memory traffic.
  - A single address tag corresponds to a large memory sector with individual valid/dirty bits for each cache line within the sector (e.g., four lines of 32 B in a 128 B sector).
- Common in GPU memory systems for managing high-bandwidth, smaller-sized data requests.



- Advantage
  - Reduced tag area.
- Drawback
  - May lead to higher miss rates compared to non-sectored caches because of constrained mapping.
- Variation
  - *Decoupled sector cache*: A design improvement where the static link between a tag and a sector is removed, allowing multiple sectors to map dynamically to improve hit rates while keeping tag costs low.

A. Seznec, “Decoupled Sectored Caches: conciliating low tag implementation cost and low miss ratio,” ISCA '94: Proceedings of the 21st annual international symposium on Computer architecture (April 1994). https://doi.org/10.1145/191995.192072.
J. B. Rothman and A. J. Smith, "Sector cache design and performance," Proceedings 8th International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems (Cat. No.PR00728), San Francisco, CA, USA, 2000, pp. 124-133, doi: 10.1109/MASCOT.2000.876437.